Milestone 5

SWE450
4/5/26

Nolan Tuttle

Grand Canyon University

College of Engineering and Technology

Software Engineering Program

Embedded Systems 2

Table of Contents

Table of Contents ..... **Error! Bookmark not defined.**

Project Demonstration Link 2

GitHub Repo Link 2

Introduction 2

System Design 3

Application Logic Design 4

Risk Matrix 4

Test Plan 5

Known Issues 7

Misc Documentation 7

Project Demonstration Link

https://youtube.com/shorts/Q4aUEeO_178?feature=share

GitHub Repo Link

https://github.com/nolantuttle/DE10AudioGame2

Introduction

This project extends prior audio output work on Intel's SoC FPGA platform by implementing a distributed interactive control system between an Adafruit ItsyBitsy 5V 32u4 microcontroller and the DE10-Standard board. In this design, user input is captured via push buttons connected to the ItsyBitsy and transmitted over a UART interface to the DE10-Standard, where button press commands trigger audio playback sequences through the onboard WM8731 audio codec. The system demonstrates reliable, real-time command communication across heterogeneous platforms and emphasizes hardware–software co-design.

The Adafruit ItsyBitsy 5V 32u4 microcontroller is responsible for monitoring push button inputs, debouncing signals, and acting as the UART transmitter during communication. Button press events are encoded and transmitted as serial data packets to ensure deterministic delivery and proper command recognition. On the DE10-Standard, the FPGA fabric implements a UART receiver interface and command decoding logic to interpret incoming button signals, while coordinating timing and triggering appropriate audio responses for interactive gameplay.

Audio output is produced through the WM8731 codec using a 16-bit I²S interface at 48 kHz. Codec configuration is performed via I²C, while the FPGA supplies the necessary clocks and manages the audio data path. By separating user input handling and control on the ItsyBitsy from real-time audio generation and game logic on the FPGA, the project highlights effective partitioning of responsibilities across embedded microcontrollers and programmable logic.

System Design

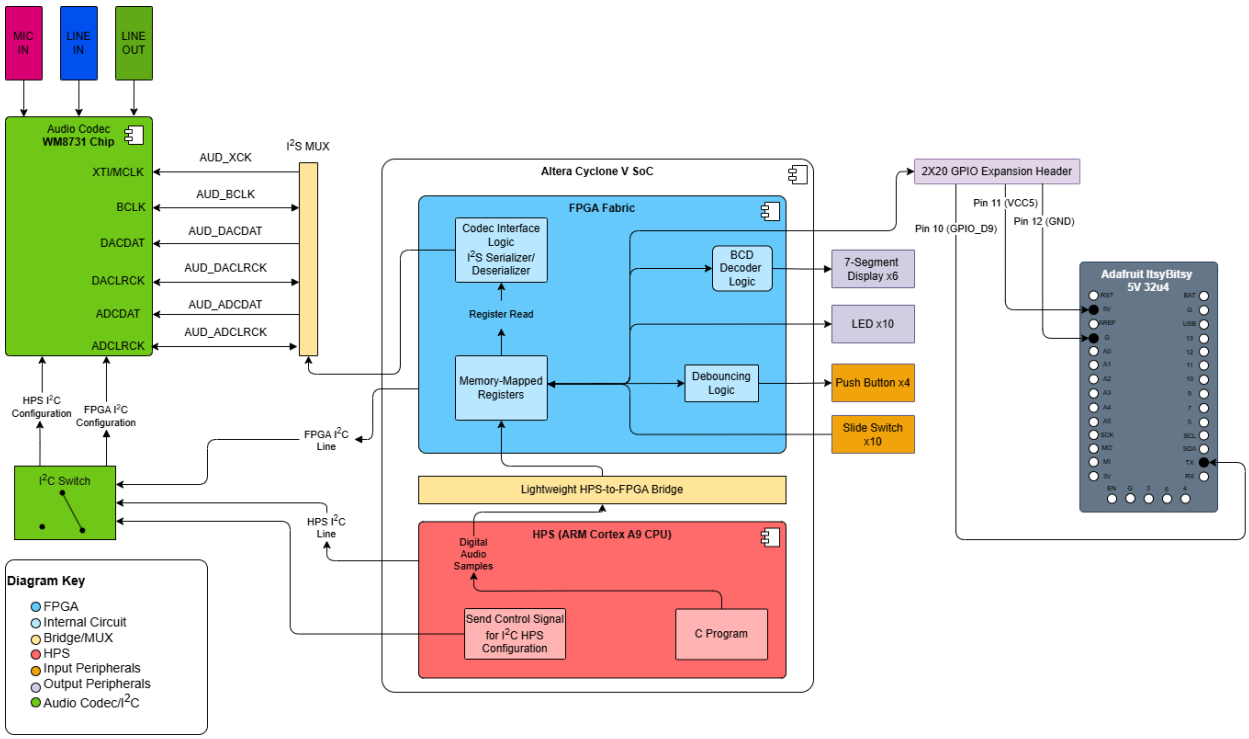

Figure 1: *System Design for ItsyBitsy Communication with DE10-Standard*

Application Logic Design

Risk Matrix

ID	Category	Description	Severity	Mitigation
1	Technical	UART data reception not synchronized with game logic clock domain	H	**Reduction** – Run UART RX on same clock domain as game logic (clock_12) to avoid clock domain crossing issues
2	Technical	Button presses dropped or misread due to UART timing	H	**Reduction** – Implement robust UART receiver with start bit detection and proper baud rate configuration (115200 baud at 12 MHz)
3	Operational	Physical connection issues between ItsyBitsy and FPGA GPIO	L	**Reduction** – Use secure header pin connections, verify continuity with multimeter before testing
4	Technical	Codec configuration failure, incorrect I2S settings	L	**Avoidance** – Use previously tested config from last semester, mimic FPGA design

Test Plan

#	Title	Summary	Steps	Data	Expected Result	Status
1	UART Byte Transmission	Verify the ItsyBitsy can transmit button press commands over UART to DE10	1. Power on DE10 and ItsyBitsy 2. Run ItsyBitsy button test code 3. Monitor FPGA LEDs for received bytes	Test bytes: 0x01, 0x02, 0x04, 0x08	FPGA LEDs display received byte values correctly	**Pass**
2	Button Press Integration	Confirm button presses trigger correct audio tones	1. Connect ItsyBitsy UART to FPGA 2. Press each button sequentially 3. Verify corresponding tone plays	Four button presses mapped to four tones	Each button press triggers unique tone (440Hz, 523Hz, 659Hz, 784Hz)	**Pass**
3	Game State Logic	Verify Simon game logic works with UART input	1. Start game on FPGA 2. Use ItsyBitsy buttons to play sequence 3. Monitor game state LEDs	Complete game sequence (3-4 rounds)	Game progresses through states correctly: GENERATE_SEQ → PLAY_SEQ → WAIT_USER → WIN/LOSE	**Pass**
4	Full Gameplay	Check complete game flow from start to multiple rounds	1. Power on DE10 and ItsyBitsy 2. Play through multiple rounds 3. Verify correct/incorrect	5+ game rounds with intentional errors	Game advances on correct sequences, enters LOSE state on incorrect input,	**Pass**

			inputs handled properly		resets properly	

Known Issues

ID	Description	Plan of Address
1	Physical KEY buttons on FPGA still active alongside UART input	Acceptable – Both input methods can coexist for testing/debugging flexibility
2	No bidirectional communication (FPGA → ItsyBitsy) implemented yet	Future work – Add UART TX module to send game state to ItsyBitsy OLED display

Misc Documentation